

Projeto em Python — Simulador Completo de Codificação de Linha

Disciplina: Comunicação de Dados

Objetivo

O objetivo deste projeto é implementar, em Python, um **simulador completo de codificação de linha**, capaz de gerar sinais digitais codificados, exibir gráficos no tempo e realizar análises de transições, componente DC e largura de banda relativa.

Este projeto **não exige relatório**. A entrega consiste apenas no código Python, nos gráficos gerados e nos resultados das análises impressos no terminal.

Codificações Obrigatórias

O simulador deve implementar, obrigatoriamente, os seguintes esquemas de codificação de linha:

1. NRZ-L (Non Return to Zero Level)

- 0 → nível baixo
- 1 → nível alto

2. NRZ-I (Non Return to Zero Invert on Ones)

- bit 1 → inverte o nível
- bit 0 → mantém o nível anterior

3. RZ (Return to Zero)

- 1 → nível alto na primeira metade do bit e zero na segunda
- 0 → zero durante todo o bit

4. Manchester

- 0 → transição alto → baixo
- 1 → transição baixo → alto

5. Manchester Diferencial

- 0 → transição no início do bit
- 1 → sem transição no início
- (todos possuem transição no meio)

6. AMI (Alternate Mark Inversion)

- $0 \rightarrow \text{nível } 0$
- $1 \rightarrow \text{alterna entre } +A \text{ e } -A$

7. Pseudoternário

- $1 \rightarrow \text{nível } 0$
- $0 \rightarrow \text{alterna entre } +A \text{ e } -A$

Requisitos Técnicos do Programa

Entrada

O programa deve aceitar as entradas:

- uma sequência de bits digitada pelo usuário (ex.: 1011001110),
- leitura de um arquivo `.txt` contendo uma linha de bits.

Funções de Codificação

Cada codificação deve ser implementada como uma função independente:

```
def encode_manchester(bits, Rb, sps):  
    """  
    bits: lista com 0 e 1  
    Rb: taxa de bits (bits/s)  
    sps: amostras por bit  
    return: t, s(t)  
    """
```

Plotagem

Para cada codificação, o programa deve:

- gerar um gráfico usando matplotlib;
- utilizar gráfico do tipo “step”;
- colocar eixo horizontal em tempo (s);
- indicar visualmente os limites de cada bit.

Análises Obrigatórias

Para cada esquema de codificação, o programa deve calcular e exibir no terminal:

- **Número de transições** no sinal;
- **Componente DC**: valor médio do sinal;
- **Estimativa da largura de banda relativa**: proporcional ao número de transições por segundo.

Exemplo esperado de saída:

```
NRZ-L: DC = -0.02, Transições = 4, Banda = baixa
Manchester: DC = 0.00, Transições = 10, Banda = alta
AMI: DC = 0.00, Transições = 6, Banda = média
```

Entrega

O grupo deve entregar:

- código Python (.py ou Jupyter);
- gráficos gerados para TODAS as codificações;
- saída do terminal com as análises de cada esquema.

Critérios de Avaliação (0–10 pontos)

Item Avaliado	Pontos
Implementação correta das 7 codificações	5,0
Plotagem e clareza visual dos gráficos	2,0
Cálculo das análises (DC, transições, banda)	2,0
Clareza, organização e modularidade do código	1,0
Total	10,0